Custo de Energia por Rodada em Aprendizado Federado

Como modelar a bateria para comparar redução de largura, de épocas e de dados

Complemento ao documento “Heterogeneidade de Modelos e Consumo de Bateria” — 18 de outubro de 2026

Objetivo. O LeanFed desconta da bateria um custo que depende só do volume de dados. Isso não serve para comparar formas diferentes de economizar energia: reduzir a **largura** do modelo, o número de **épocas** ou a fração de **dados**. Este documento propõe um modelo de energia por rodada que trata os três ajustes da mesma forma, mostra o que cada um economiza e como calibrar os números.

Conteúdo

- 1. Modelo de energia de uma rodada
- 2. O que cada ajuste reduz
- 3. Exemplo numérico
- 4. Números para a CNN do projeto
- 5. Atualizar a bateria e escolher o ajuste
- 6. Calibração
- 7. Esboço de código
- 8. Origem e limitações do modelo
- 9. Referências

1. Modelo de energia de uma rodada

A energia que o cliente *e* gasta em uma rodada é separada em três partes. Com largura do submodelo p, L épocas locais e fração de dados η:

$$E_e(p, L, \eta) = A_e \cdot f(p) \cdot \eta \cdot L \; + \; C_e \cdot s(p) \; + \; D_e$$

O primeiro termo é a **computação**, o segundo a **comunicação** e o terceiro o **custo fixo** de participar da rodada. A seção 8 explica a origem de cada parte e as limitações do modelo.

Termo	Significado
A_e	Energia de **uma época** sobre todos os dados locais com o modelo **completo**.
f(p) = F(p)/F(1)	Custo de computação relativo da largura p (FLOPs ou MACs por amostra).

Termo	Significado
C_e	Energia para baixar e enviar o modelo **completo**. Depende da rede do dispositivo (Wi-Fi, 4G); enviar costuma gastar mais que baixar.
s(p) = S(p)/S(1)	Tamanho relativo do submodelo (número de parâmetros).
D_e	Custo fixo de participar: acordar o dispositivo, carregar o modelo, a “cauda” do rádio depois de transmitir.

Os dois coeficientes principais podem ser escritos a partir de grandezas físicas:

$$A_e = k_e \cdot 3F(1) \cdot n_e \qquad C_e = \left(\varepsilon_e^{\downarrow} + \varepsilon_e^{\uparrow}\right) \cdot 4 \cdot S(1)$$

- k_e: energia por operação (MAC) no hardware do dispositivo.
- O fator 3 aproxima o custo do treino: forward + backward ≈ 3 × o forward.
- n_e: número de amostras locais de treino.
- $\varepsilon^{\downarrow}_e$, $\varepsilon^{\uparrow}_e$: energia por byte recebido e enviado; o fator 4 é o número de bytes por parâmetro em float32.

> **Por que normalizar:** os coeficientes estão sempre definidos para o modelo completo, e os fatores relativos f(p) e s(p) fazem o ajuste para cada largura. Assim, o mesmo A_e e C_e servem para todas as larguras, e só as tabelas de f e s dependem da arquitetura.

2. O que cada ajuste reduz

Ajuste	Computação	Comunicação	Custo fixo
Dados η	∝ η	não muda	não muda
Épocas L	∝ L	não muda	não muda
Largura p	∝ f(p) ≈ p^2	∝ s(p) ≈ p^2	não muda
Pular a rodada	0	0	0

Daí saem três consequências, que podem virar hipóteses da dissertação:

Consequência 1 — Em energia, dados e épocas são equivalentes

Só o produto η·L (o total de amostras processadas) importa. A diferença entre eles é **estatística**, não energética: várias épocas sobre poucos dados não aprendem o mesmo que uma época sobre todos os dados. Por isso a comparação entre os ajustes deve ser feita **com a mesma energia gasta**, medindo a precisão (comparação “iso-energia”).

Consequência 2 — Largura é o único ajuste que reduz a comunicação

Reduzindo dados ou épocas, o cliente ainda baixa e envia o modelo inteiro, então nunca gasta menos que $C_e + D_e$ por rodada. Com bateria muito baixa, ele precisa pular rodadas. Com a largura, o mínimo cai para $C_e \cdot s(p_{min}) + D_e$, e o cliente consegue continuar participando com pouca energia.

Consequência 3 — Quem vence depende de C_e/A_e

C_e/A_e mede o peso da comunicação em relação à computação. Se a comunicação pesa mais (modelo grande, rede ruim), a largura tende a vencer. Se a computação pesa mais, os três ajustes economizam parecido, e a qualidade do aprendizado decide.

3. Exemplo numérico

Valores **hipotéticos**, em unidades normalizadas, só para ilustrar: $A_e = 1$, $C_e = 0{,}5$, $D_e = 0{,}1$, $L = 1$, com os fatores f(p) e s(p) da CNN do projeto (seção 4). Uma rodada completa (p = 1, η = 1) custa 1 + 0,5 + 0,1 = 1,6.

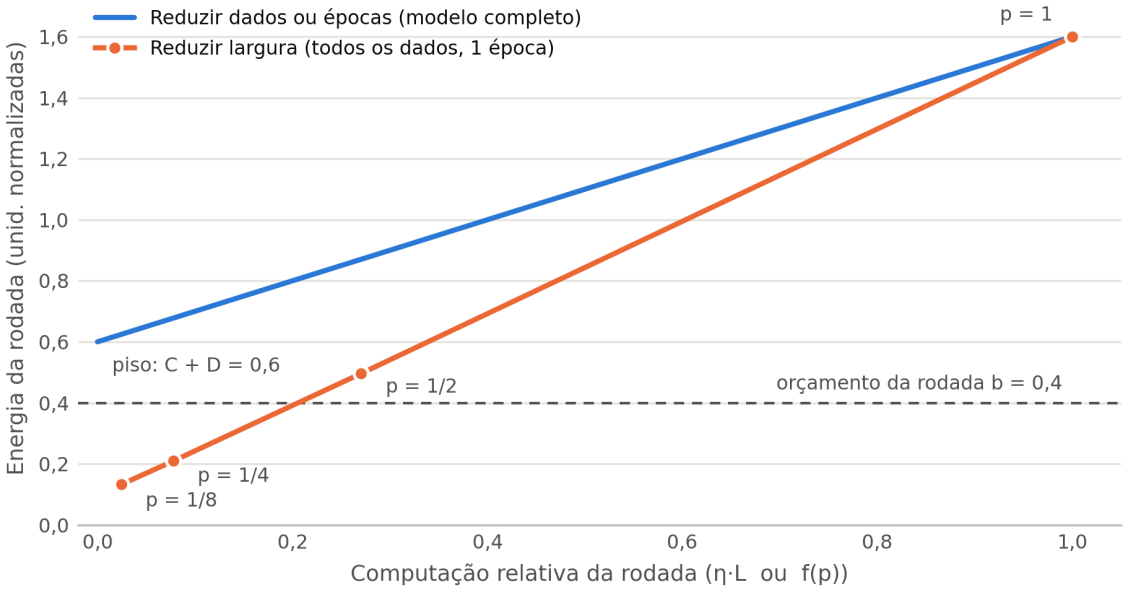


Figura 1 — Energia de uma rodada em função da computação feita. A reta azul reduz dados ou épocas com o modelo completo; a curva laranja reduz a largura usando todos os dados. A reta tracejada é um orçamento de 0,4 por rodada. Reduzindo só dados ou épocas, a energia nunca fica abaixo de C + D = 0,6; reduzindo a largura, ela se aproxima de D = 0,1.

Orçamento b	Só dados (p = 1)	Só largura (η = 1)
0,8	η* = (0,8 − 0,5 − 0,1) / 1 = **0,2**: modelo inteiro com 20% dos dados	p = 1/2 custa 0,270 + 0,126 + 0,1 = 0,496 ≤ 0,8 → **p = 1/2** com todos os dados
0,4	η* = (0,4 − 0,6) / 1 < 0 → **não cabe**: o cliente pula a rodada	p = 1/4 custa 0,078 + 0,032 + 0,1 = 0,210 ≤ 0,4 → **p = 1/4**, continua participando

Com orçamento 0,8, a pergunta de pesquisa aparece de forma concreta: com a mesma energia, o que contribui mais para o modelo global — 20% dos dados com o modelo inteiro, ou todos os dados com metade da largura? Com orçamento 0,4, só a largura permite participar.

4. Números para a CNN do projeto

Contagem para o `Net` atual de `fedavg_app/task.py` (entrada 32×32), conferida executando o código da seção 7. No forward, são **10,85 M MACs** por amostra; o modelo tem **620 mil parâmetros** (2,5 MB em float32).

Camada	MACs	% MACs	Parâmetros	% parâm.	Escala com
`conv1` (3→32)	884.736	8,2%	896	0,1%	p
`conv2` (32→64)	4.718.592	43,5%	18.496	3,0%	p^2
`conv3` (64→128)	4.718.592	43,5%	73.856	11,9%	p^2
`fc1` (2048→256)	524.288	4,8%	524.544	84,6%	p^2
`fc2` (256→10)	2.560	0,0%	2.570	0,4%	p
Total	**10.848.768**	100%	**620.362**	100%	

Largura p	f(p) — computação	s(p) — comunicação
1	100%	100%
1/2	27,0%	25,2%
1/4	7,8%	6,4%
1/8	2,5%	1,6%

O valor não é exatamente p^2 porque a `conv1` (3 canais de entrada fixos) e a `fc2` (10 saídas fixas) escalam só com *p*.

> **Assimetria importante:** a `fc1` concentra **85% dos parâmetros**, mas só **5% da computação**; as convoluções concentram quase toda a computação, mas poucos parâmetros. Ou seja, nesse modelo a comunicação e a computação vêm de camadas diferentes, e a proporção C_e/A_e muda bastante de arquitetura para arquitetura. Vale testar pelo menos dois modelos (por exemplo, esta CNN e uma ResNet-18, que é dominada por convoluções).

5. Atualizar a bateria e escolher o ajuste

5.1 Atualização

A cada rodada em que o cliente participa:

$$B_e \leftarrow B_e - E_e(p, L, \eta)$$

Para mais realismo, desconte também um consumo pequeno em repouso (δ_e) nas rodadas em que ele não participa e, ao usar traces reais, some a recarga quando o dispositivo estiver carregando.

5.2 Escolha do ajuste

O servidor parte do orçamento da rodada e inverte a fórmula:

$$b = \frac{B_e}{\text{rodadas restantes}}$$

- **Só dados** (LeanFed corrigido — o original ignora C_e e D_e):

$$\eta^* = \frac{b - C_e - D_e}{A_e \cdot L}$$

- **Só épocas:**

$$L^* = \left\lfloor \frac{b - C_e - D_e}{A_e \cdot \eta} \right\rfloor$$

Épocas são números inteiros, então a granularidade é ruim. Considere usar **passos locais** (número de batches) no lugar de épocas.

- **Só largura:** a maior p tal que $E_e(p, L, \eta) \leq b$.
- **Combinado:** testar as combinações de (p, η) que cabem no orçamento e escolher por uma regra — por exemplo, reduzir a largura até p_{min} e só depois cortar dados.

Nos três primeiros casos, se nada cabe no orçamento, o cliente pula a rodada. Quando pula, a energia não gasta aumenta o orçamento das rodadas seguintes.

6. Calibração

6.1 O problema da não-linearidade

O modelo $k_e \cdot f(p)$ supõe que a energia por operação é constante. Na prática, modelos pequenos aproveitam mal a GPU (overhead de lançamento de kernels, unidades ociosas). Por isso $p = 1/4$ economiza **menos** que os 7,8% da tabela sugerem. A solução é medir a energia por amostra em cada largura e usar a tabela medida $f_{med}(p)$ no lugar de f(p).

6.2 Opção (a): fórmula analítica com constantes medidas — recomendada

Faça uma etapa de perfilamento separada, fora dos experimentos:

1. Meça a potência ociosa da GPU por alguns segundos.
2. Para cada largura p, rode **um único** processo de treino por um número fixo de batches.
3. Leia o contador de energia do NVML antes e depois (`nvmlDeviceGetTotalEnergyConsumption`, em mJ, disponível em GPUs a partir da geração Volta).
4. Desconte potência ociosa × tempo e divida pelo número de amostras: energia por amostra e(p).
5. Repita algumas vezes e use a média; $f_{med}(p) = e(p)/e(1)$ entra na fórmula no lugar de f(p).

O resultado é determinístico durante os experimentos e, portanto, reproduzível. Os parâmetros de comunicação (energia por byte, banda) não podem ser medidos na GPU: tire-os de medições publicadas para smartphones ou de medição própria, e passe-os por cliente no `--node-config`.

6.3 Opção (b): potência × tempo emulado

$$E_{comp} = P_e \cdot t_{sim} \qquad E_{comm} = P_e^{\text{rádio}} \cdot \frac{\text{bytes}}{\text{banda}_e}$$

Reaproveita o `sim_time` que o `train()` já calcula e captura a não-linearidade sozinha. O problema é que, no setup atual, **todos os clientes dividem a mesma GPU**: o tempo medido inclui a disputa com os outros containers, varia entre execuções e atrapalha a reprodutibilidade.

> **Atenção:** o NVML não separa a energia de cada container. Medições feitas durante um experimento com vários clientes misturam o consumo de todos; por isso a calibração deve ser feita em perfilamento separado, uma configuração por vez.

7. Esboço de código

7.1 Contagem de MACs e parâmetros para qualquer largura

Usa hooks do PyTorch, então funciona para qualquer arquitetura feita de `Conv2d` e `Linear`. Os números da seção 4 saíram desta função.

```
def macs_and_params(width: float) -> tuple[int, int]:
    net, macs = Net(width), 0
    def hook(m, inp, out):
        nonlocal macs
        if isinstance(m, nn.Conv2d):
            kh, kw = m.kernel_size
            macs += out.numel() * (m.in_channels // m.groups) * kh * kw
        elif isinstance(m, nn.Linear):
            macs += m.in_features * m.out_features
    hs = [m.register_forward_hook(hook) for m in net.modules()
          if isinstance(m, (nn.Conv2d, nn.Linear))]
    net(torch.zeros(1, 3, 32, 32))
    for h in hs:
        h.remove()
    return macs, sum(t.numel() for t in net.parameters())
```

7.2 Energia prevista de uma rodada (planejamento no servidor)

```
def round_energy(prof: dict, width: float, epochs: int, frac: float) -> float:
    f = REL_COMPUTE[width]   # f(p), ou f_med(p) medido na calibração
    s = REL_PARAMS[width]    # s(p)
    return prof["A"] * f * frac * epochs + prof["C"] * s + prof["D"]
```

7.3 Escolha da largura dentro do orçamento

```
WIDTHS = (1.0, 0.5, 0.25, 0.125)

def choose_width(prof: dict, battery: float, rounds_left: int, epochs: int = 1) -> float:
    budget = battery / rounds_left
    for p in WIDTHS:                                    # da maior para a menor
        if round_energy(prof, p, epochs, 1.0) <= budget:
            return p
    return 0.0                                          # não participa nesta rodada
```

7.4 Consistência entre planejamento e execução

No cliente, o desconto feito batch a batch deve usar os mesmos coeficientes:

- por batch: $k_e \cdot 3F(p) \cdot$ tamanho do batch;
- ao baixar e ao enviar o modelo: a parte correspondente de $C_e \cdot s(p)$;
- uma vez por rodada: D_e.

Assim, o que o servidor planeja e o que o cliente gasta seguem o mesmo modelo, e a diferença entre os dois vem só do que você introduzir de propósito — por exemplo, ruído na estimativa da bateria para testar a robustez da política.

8. Origem e limitações do modelo

O modelo não foi tirado de um artigo específico. Ele junta uma estrutura padrão da literatura de FL com restrição de energia (computação + comunicação) com dois acréscimos: os fatores de largura f(p) e s(p) e o custo fixo D_e. Esta seção mostra de onde vem cada parte, o que é suposição e como validar.

8.1 A base: computação + comunicação

A decomposição aparece, por exemplo, em Yang et al. [1], equações (2), (3) e (5) do artigo (conferidas no PDF):

$$E_k^C = \kappa\, I_k\, C_k\, D_k\, f_k^2 \qquad\qquad E_k^T = t_k\, p_k, \quad t_k\, r_k \geq s$$

- **Computação:** κ é uma constante do chip, I_k o número de iterações locais, C_k os ciclos de CPU por amostra, D_k o número de amostras e f_k a frequência da CPU.
- **Transmissão:** potência × tempo, onde o tempo é o tamanho do modelo *s* dividido pela taxa r_k.

O modelo deste documento é a mesma estrutura, reescrita:

Yang et al. [1]	Este documento
I_k (iterações locais)	*L* (épocas)
D_k (amostras)	$\eta \cdot n_e$
C_k (ciclos por amostra)	$\propto 3F(p)$ — é aqui que entra a largura
$\kappa\, f_k^2$ (energia por ciclo)	k_e

Yang et al. [1]	Este documento
$p_k \cdot s / r_k$	$C_e \cdot s(p)$ — energia por byte × tamanho do modelo

Detalhe relevante: Yang et al. assumem que o tamanho do modelo é **igual para todos os usuários** (“the dimensions of the local FL model are fixed for all users”). Com submodelos, o *s* passa a depender da largura — é exatamente a lacuna explorada aqui. Eles também desprezam o download, supondo que a estação base transmite com muita potência; este documento inclui o download, mas ele pode ser desprezado com a mesma justificativa.

8.2 O que foi acrescentado

Peça	De onde veio	Grau de confiança
f(p) e s(p)	As mesmas métricas de custo do HeteroFL [5]: FLOPs como medida de computação e número de parâmetros como medida de comunicação, aqui divididas pelo valor do modelo completo (detalhes em 8.3)	A contagem é exata. A suposição **energia ∝ FLOPs** é fraca (8.4, item 1)
Fator 3 (treino ≈ 3× forward)	Aproximação padrão: o backward custa cerca de 2× o forward	Boa aproximação; irrelevante após normalizar, pois se cancela em f(p)
Custo fixo D_e	Medições em celulares: o rádio permanece em alta potência após cada transferência (“tail energy”) [2]	Fenômeno bem documentado em 3G/GSM; o **valor** em redes atuais é desconhecido
Normalização	Escolha de modelagem, para separar o que é do dispositivo (A_e, C_e) do que é da arquitetura (f, s)	É só reescrita; não muda o modelo
Valores do exemplo (A = 1, C = 0,5, D = 0,1)	Inventados, só para ilustrar	Nenhum — não usar como resultado

8.3 f(p) e s(p): as métricas de custo do HeteroFL

Como o HeteroFL cria os submodelos. O HeteroFL [5] reduz a **largura** das camadas ocultas (o número de canais), mantendo a profundidade. Para uma camada oculta com pesos W_g de dimensão $d_g \times k_g$ (canais de saída × canais de entrada), o submodelo do nível *l* mantém uma fração r^{l-1} dos canais de saída **e** de entrada. Como as duas dimensões encolhem, o número de pesos cai com o quadrado (seção 3.1 do artigo):

$$|W_l| = r^{2(l-1)}\,|W_g|$$

Os experimentos usam r = 0,5 e cinco níveis, de ‘a’ (modelo completo) a ‘e’, com larguras relativas 1; 0,5; 0,25; 0,125 e 0,0625.

> **Cuidado com a notação:** no HeteroFL, a letra *p* é o **índice do nível** e *r* é a razão de encolhimento. Neste documento, *p* é a **própria largura relativa**: o nível *l* do HeteroFL corresponde a $p = r^{l-1}$, e a lei acima vira $s(p) \approx p^2$.

Como o HeteroFL mede o custo. As tabelas de resultados trazem, para cada modelo ou combinação de modelos, três colunas de custo: **Parameters** (número de parâmetros), **FLOPs** (operações de um forward) e **Space (MB)** (memória ocupada pelos parâmetros). Há também uma coluna **Ratio**, com o número de parâmetros relativo ao maior modelo da combinação. Os autores dizem usar essas colunas para indicar os requisitos de computação e de comunicação dos métodos. Exemplo da Tabela 1 do artigo (MNIST), comparando o modelo completo com o de meia largura:

Modelo	Largura	Parameters	FLOPs	Space (MB)
‘a’	1	1,6 M	80,5 M	5,94
‘b’	0,5	391 K	20,5 M	1,49
‘b’ / ‘a’		≈ 0,24	≈ 0,25	≈ 0,25

Reduzir a largura à metade reduz parâmetros e FLOPs a cerca de 1/4, como prevê a lei quadrática.

Como isso vira f(p) e s(p). As duas funções são essas mesmas colunas, divididas pelo valor do modelo completo:

$$f(p) = \frac{\text{FLOPs}(p)}{\text{FLOPs}(1)} \qquad s(p) = \frac{\text{Parâmetros}(p)}{\text{Parâmetros}(1)}$$

- **f(p)** representa a computação: FLOPs por amostra no forward. O treino custa cerca de 3× isso, mas o fator se cancela na divisão.
- **s(p)** representa a comunicação: em cada rodada o cliente baixa e envia exatamente os parâmetros do seu submodelo, então os bytes transmitidos são proporcionais ao número de parâmetros (4 bytes cada em float32). A coluna Space (MB) do HeteroFL é essa mesma grandeza em megabytes.

Por que não dá exatamente p^2 no modelo inteiro. A lei quadrática vale para camadas ocultas, em que entrada e saída encolhem juntas. A primeira camada tem entrada fixa (3 canais de cor) e a última tem saída fixa (o número de classes), então essas duas escalam só com *p*. Na CNN do projeto, isso dá $s(1/2) = 25{,}2\%$ e $f(1/2) = 27{,}0\%$ em vez de 25% (seção 4). Por isso o documento usa a **contagem exata** (código da seção 7.1) em vez da fórmula p^2.

O que o HeteroFL não faz. O artigo reporta FLOPs e parâmetros como indicadores de custo, mas não mede tempo nem energia. O passo de “FLOPs e parâmetros” para “joules” é o que este modelo acrescenta, com os coeficientes A_e e C_e, e é exatamente esse passo que pode falhar (item 1 de 8.4) e que a calibração verifica.

8.4 Onde o modelo pode falhar

1. **Energia proporcional a FLOPs é a maior fraqueza.** Yang, Chen e Sze [3] mostram que reduzir computação ou tamanho do modelo não reduz a energia na mesma proporção, porque boa parte dela vai para mover dados na memória. Logo, f(p) provavelmente **superestima** a economia de submodelos pequenos. Por isso a calibração (tabela medida $f_{med}(p)$, seção 6) é parte essencial do modelo, não um detalhe.
2. **Frequência da CPU fixa.** Yang et al. [1] otimizam a frequência f_k, e a energia por ciclo cresce com f_k^2. Tratar k_e como constante equivale a supor que o dispositivo sempre roda na mesma frequência.
3. **Potência de base durante a rodada.** Enquanto treina, o dispositivo gasta uma potência de base (CPU acordada, memória) proporcional ao **tempo**, não às operações. Se o tempo cresce com o trabalho, essa parte fica embutida em A_e; se não cresce, aparece como custo fixo extra. Medir de ponta a ponta resolve isso automaticamente.
4. **Bateria linear.** O modelo supõe que a carga diminui exatamente pela energia consumida. Baterias reais têm efeitos não-lineares (perdem capacidade com corrente alta, se recuperam em repouso, variam com a temperatura). A maioria dos artigos de FL ignora isso; para um mestrado é uma simplificação aceitável, desde que declarada.

> **A conclusão qualitativa resiste a essas limitações:** reduzir dados ou épocas não diminui a comunicação nem o custo fixo, então existe um piso de C + D. Isso vale para quaisquer C, D > 0, sem depender de números. O que muda com os itens 1 a 4 é **o quanto** a largura vence.

8.5 Como validar

Isso pode virar uma seção da dissertação (“validação do modelo de energia”). Com o perfilamento por NVML da seção 6, teste três previsões:

1. **Energia × fração de dados η** (com p = 1): a fórmula prevê uma **reta**. A inclinação estima A_e e o intercepto estima o custo fixo.
2. **Energia × épocas L:** também uma reta, com a mesma inclinação por amostra processada.
3. **Energia × largura p:** compare a energia medida com f(p). Se a razão $f_{med}(p)/f(p)$ estiver longe de 1 para *p* pequeno, a fraqueza do item 1 está confirmada, e a tabela medida substitui f(p).

Faça uma regressão linear e reporte o R^2. Se as retas se ajustarem bem, o modelo está **validado empiricamente no hardware usado**, o que é mais do que LeanFed [4] e EAFL [6] fazem. A comunicação não pode ser medida na GPU: cite medições da literatura, mantenha C_e como parâmetro e faça uma análise de sensibilidade, variando C_e/A_e e mostrando como a conclusão muda.

9. Referências

[1] Z. Yang, M. Chen, W. Saad, C. S. Hong, M. Shikh-Bahaei. Energy Efficient Federated Learning Over Wireless Communication Networks. IEEE Transactions on Wireless Communications. https://arxiv.org/abs/1911.02417

[2] N. Balasubramanian, A. Balasubramanian, A. Venkataramani. Energy Consumption in Mobile Phones: A Measurement Study and Implications for Network Applications. ACM IMC 2009. https://people.cs.umass.edu/~arun/papers/TailEnder.pdf

[3] T.-J. Yang, Y.-H. Chen, V. Sze. Designing Energy-Efficient Convolutional Neural Networks Using Energy-Aware Pruning. CVPR 2017. https://openaccess.thecvf.com/content_cvpr_2017/html/Yang_Designing_Energy-Efficient_Convolutional_CVPR_2017_paper.html

[4] R. Pereira, C. J. Vaca-Rubio, L. Blanco. Learn More by Using Less: Distributed Learning with Energy-Constrained Devices (LeanFed). https://arxiv.org/abs/2412.02289

[5] E. Diao, J. Ding, V. Tarokh. HeteroFL: Computation and Communication Efficient Federated Learning for Heterogeneous Clients. ICLR 2021. https://arxiv.org/abs/2010.01264

[6] EAFL (Arouj & Abdelmoniem, 2022) e demais trabalhos citados — ver o documento “Heterogeneidade de Modelos e Consumo de Bateria”.